

SYSTEM DESIGN

DOCUMENT

Multimodal RAG Agentic Knowledge AI Assistant

100% Open-Source - Local GGUF Mistral - Cloud-Native Infra

Architecture - Data Flow - Tooling - Security - Observability

Reverse-engineered from 49 production source files

Version 2.0 - Phase 24 of 31

1. Executive Summary

This document describes the production architecture of the **Multimodal RAG Agentic Knowledge AI Assistant** — a fully open-source, privacy-preserving question-answering system that ingests text, PDF, Word, Excel, image, audio, and video; reasons over a hybrid knowledge base; and decides autonomously between retrieval-augmented generation, live web search, conversational memory, or direct LLM answering.

Unlike the original plan, the implemented system is **100% open-source at the model layer** (local GGUF Mistral-7B-Instruct via llama.cpp), uses an **agentic router** to choose the answering strategy per query, and is built on managed free-tier cloud infrastructure (Qdrant Cloud, Upstash Redis, MongoDB Atlas) so it can be deployed without GPU servers or paid LLM APIs.

1.1 Key Differences from Original Plan

Area	Original Plan	Implemented (Actual)
LLM	Hosted API model	Local GGUF Mistral-7B-Instruct (llama.cpp), fully offline
Answering	RAG only	Agentic: RAG / Web / Memory / Direct / Hybrid routing
Web access	None	Tavily web-search tool with SSRF + injection guards
Modalities	Text + PDF	Text, PDF, Word, Excel, Image, Audio, Video
Retrieval	Vector only	Hybrid: BM25 (sparse) + Qdrant (dense) + cross-encoder rerank
Memory	Single store	Redis (short-term) + MongoDB (long-term summaries)
Folder layout	Plan layout	Restructured into domain modules under app/
Cost	Paid APIs	\$0 — open models + managed free tiers

2. System Context & Goals

2.1 Functional Goals

- Accept queries and documents across 7 modalities and return grounded answers.
- Autonomously route each query to the optimal strategy (knowledge base, web, memory, or direct).
- Preserve conversational context across turns and sessions (multi-store memory).
- Decompose complex/multi-hop questions into retrievable sub-queries.
- Fuse, deduplicate, and rerank evidence while detecting contradictions.
- Operate fully offline at the model layer for privacy and zero API cost.

2.2 Non-Functional Goals (Production Bar)

Attribute	Target	Mechanism in Code
Latency	< 1s retrieval, streamed gen	Embedding cache, batched encode, async wrappers
Resilience	No single point of failure	Per-service circuit breakers + graceful fallback
Observability	Full request trace	OpenTelemetry spans + Prometheus on every stage
Security	Hostile-input safe	Injection sanitization, SSRF guard, PII scrub
Privacy	GDPR-capable	gdpr_purge across Qdrant/Redis/Mongo

Attribute	Target	Mechanism in Code
Reproducibility	Deterministic ingest	SHA-256 keys, pinned model file + checksum
Cost	\$0 infra	Open models + managed free tiers

3. High-Level Architecture

The system is organized into eight cooperating layers. A query enters through the API layer, is normalized and sanitized, routed by the agent layer to an execution plan, executed against retrieval / web / memory tools, fused and reranked, then answered by the local LLM and persisted to memory.

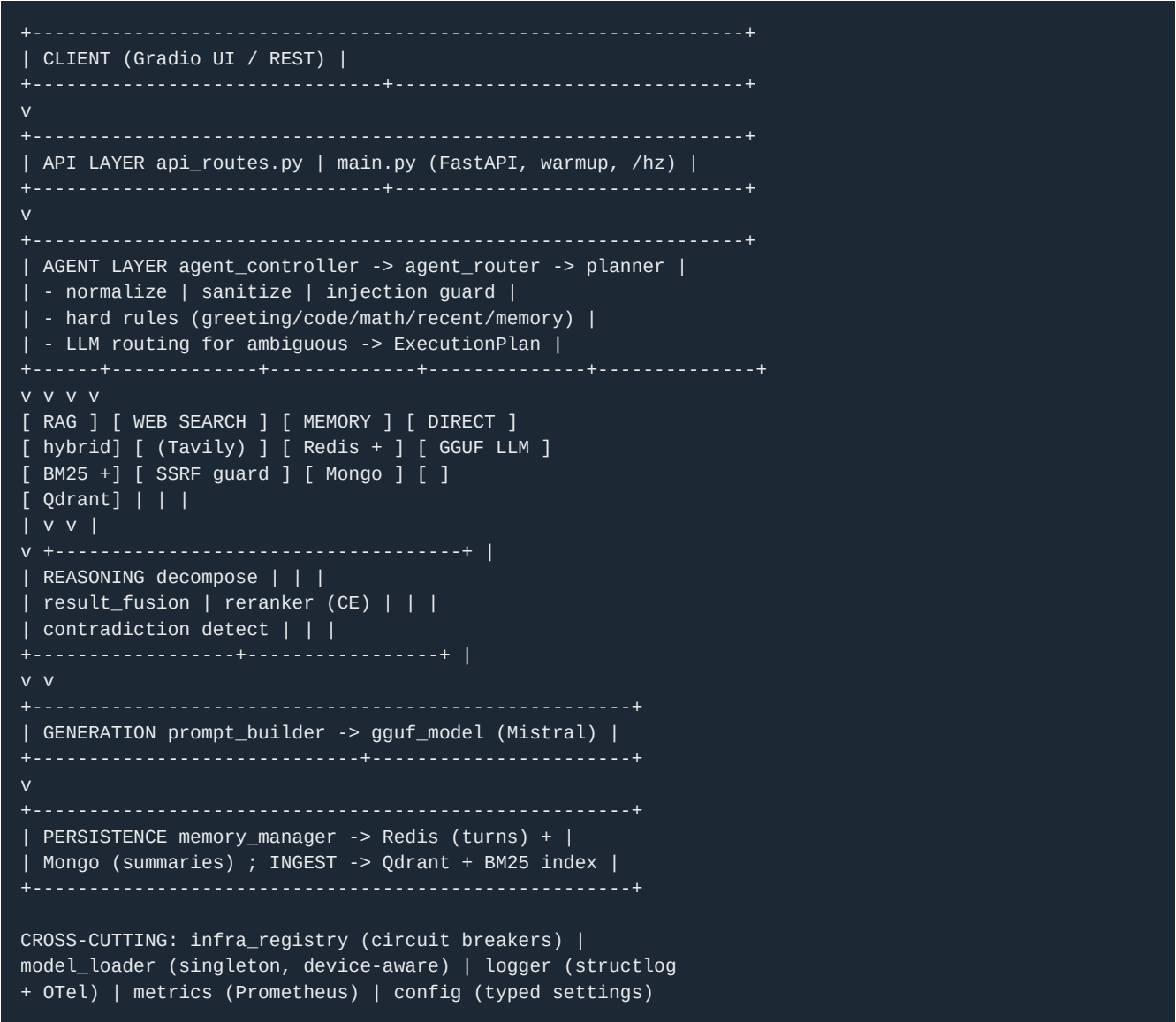


Figure 1 — Layered runtime architecture (derived from source)

4. Component Inventory (As-Built)

Every module below exists in the analyzed codebase. Grouped by domain.

4.1 Agent & Orchestration

Module	Responsibility
agent_controller.py	Entry point; normalize, sanitize, timeout, fallback, confidence, trace
agent_router.py	Hard-rule + LLM routing into rag/search/direct/memory/hybrid
agent_schema.py	Pydantic contracts: AgentDecision, ExecutionPlan, ToolCall, AgentResponse
planner.py	Builds an ExecutionPlan (steps, costs, dependency ordering, step cap)
agent_executor.py	Executes the plan steps (orchestrates tools)
tool_registry.py	Typed registry of callable tools with validated I/O

4.2 Retrieval & Reasoning

Module	Responsibility
retriever.py	Dense vector retrieval from Qdrant
bm25_retriever.py	Sparse lexical retrieval (BM25 index, pickled)
hybrid_retriever.py	Fuses sparse + dense candidate sets
reranker.py	Cross-encoder rerank, MMR diversity, modality boost, low-conf flag
query_decomposer.py	Detects comparative/temporal/multi-hop; splits into sub-queries
result_fusion.py	Score fusion, dedup, contradiction detection, cross-modal linking
reasoning_engine.py	Multi-step reasoning & final context assembly

4.3 Embeddings & Ingestion (Multimodal)

Module	Responsibility
text_embedder.py	SentenceTransformer; batched, cached, Matryoshka + sparse proxy
clip_text_embedder.py	CLIP text tower for cross-modal alignment
image_embedder.py	CLIP visual embeddings
multimodal_embedder.py	Orchestrates text + image embedding spaces
document_ingest.py / text_ingest.py	PDF / Word / Excel / plaintext extraction + chunk
image_ingest.py / frame_captioner.py	Image load (EXIF, resize) + BLIP captioning
audio_ingest.py	Whisper ASR transcription
video_ingest.py / video_frames.py	Keyframe extraction + per-frame caption/OCR

4.4 Memory, LLM & Tools

Module	Responsibility
redis_memory.py	Short-term turns + embedding/caption cache (Upstash)
mongo_memory.py	Long-term rolling summaries (MongoDB Atlas)
memory_manager.py	Coordinates Redis + Mongo read/write
memory_filter.py	Semantic + recency + importance filtering of history
memory_fusion.py	Budget-allocated context (summary/history/vector)

Module	Responsibility
summarizer.py	Incremental conversation compression to Mongo
gguf_model.py	Local Mistral-7B-Instruct via llama.cpp, streaming
model_loader.py	Singleton loader; device detect, warmup, timeout, retry
download_model.py	HF Hub fetch; atomic write, disk guard, checksum
web_search.py	Tavily search; SSRF guard, blocklist, injection scrub, summarize

4.5 Cross-Cutting Infrastructure

Module	Responsibility
config.py	Typed settings (model paths, dims, thresholds, cloud creds)
infra_registry.py	Lazy singletons + per-service circuit breakers + GDPR purge
metrics.py	Prometheus exporters (noop-safe when disabled)
logger.py	structlog JSON logging + OpenTelemetry trace correlation
api_routes.py / main.py	FastAPI routes, lifespan warmup, health/readiness

5. Core Data Flows

5.1 Ingestion Pipeline

```
upload -> router.py (detect modality)
|-- text/pdf/word/excel -> document_ingest / text_ingest
|-- image -> image_ingest -> frame_captioner (BLIP)
|-- audio -> audio_ingest (Whisper ASR)
'-- video -> video_ingest -> video_frames -> cap/OCR
v
chunker.py (normalize, chunk_id, token estimate)
v
text_embedder / multimodal_embedder (batched, cached, dedup)
v
qdrant_store.upsert + BM25 index update
v
ingestion_pipeline.py (orchestrates, traces, metrics)
```

5.2 Query Pipeline (Agentic)

```
query -> agent_controller.handle()
normalize -> sanitize(injection) -> length cap
v
agent_router.route()
hard rules? -> greeting/code/math -> DIRECT
recent/news -> SEARCH
memory reference -> MEMORY
else LLM route -> {rag | search | memory | hybrid}
v
planner.create_plan() -> ExecutionPlan(steps, costs, deps, cap)
v
agent_executor runs steps:
decompose -> (bm25 + qdrant) -> result_fusion -> reranker
v
memory_fusion (summary + history + vector) -> prompt_builder
v
gguf_model.generate (Mistral, streamed)
v
formatter -> response.py -> persist (Redis turn, Mongo summary)
```

6. Updated Tool & Technology Stack

This table reflects what the **code actually imports and uses** — the authoritative stack for deployment, requirements, and the dev-team skills.

6.1 Models (100% Open-Source)

Capability	Model / Library	Source
Generation LLM	Mistral-7B-Instruct-v0.2 GGUF (Q4_K_M)	TheBloke @ HF Hub, llama.cpp
Text embeddings	SentenceTransformers model	sentence-transformers
Cross-modal	CLIP (text + vision towers)	transformers (CLIPModel)
Image caption	BLIP conditional generation	transformers (Blip*)
Speech-to-text	faster-whisper (Whisper)	faster-whisper
Reranker	CrossEncoder	sentence-transformers CrossEncoder

6.2 Data & Infrastructure (Managed Free Tier)

Concern	Technology	Role
Vector DB	Qdrant Cloud	Dense vector store + payload filters
Lexical index	BM25 (rank_bm25, pickled)	Sparse retrieval for hybrid
Short-term memory	Upstash Redis Cloud	Conversation turns + caches
Long-term memory	MongoDB Atlas	Rolling conversation summaries
Web search	Tavily API	Live information retrieval tool
Model registry	Hugging Face Hub	GGUF model download (checksummed)

6.3 Frameworks & Libraries

Layer	Stack
API / Service	FastAPI, Uvicorn, Pydantic v2
LLM runtime	llama-cpp-python (GGUF), GGUF Mistral
Orchestration	LangChain (retrievers/runnables), custom agent layer
Vision/Audio	Pillow, OpenCV (frames), faster-whisper, transformers
Resilience	tenacity (retry), pybreaker (circuit breaker)
Observability	OpenTelemetry, Prometheus client, structlog
Security/Privacy	Presidio (PII), custom injection + SSRF guards
Eval (planned P25)	Ragas, deepeval/LangSmith datasets
UI (planned P28)	Gradio
MLOps (planned P29)	GitHub Actions, Docker, MLflow
Deploy (planned P30)	AWS (ECR/ECS or EC2), Terraform/IaC
Monitoring (P31)	Prometheus + Grafana, OTel collector

7. Security & Privacy Architecture

Security is layered. Untrusted input enters from three surfaces — user query, ingested documents, and web-search results — and each is independently defended.

Surface	Threat	Control (in code)
User query	Prompt injection	Pattern strip in controller, router, embedder
Documents	Injected instructions in OCR/caption	frame_captioner + text_embedder sanitize
Web results	Malicious content / SSRF	web_search SSRF block + domain blocklist + scrub
Memory	PII leakage into prompt	Presidio scrub in filter/formatter/summarizer
Vector DB	Cross-tenant leakage	Qdrant payload filters (per session/user)
Secrets	Credential exposure	Typed config from env; no hardcoded keys
Right to be forgotten	GDPR compliance	infra_registry.gdpr_purge across all stores

Hardening backlog (Phase 26 — Guardrails): add an output-side validation/guardrail layer (jailbreak classifier, schema-constrained answers, toxicity & PII egress checks) and signed audit log of every routing decision.

8. Observability & Reliability

- **Tracing:** OpenTelemetry spans wrap every stage (route, plan, retrieve, rerank, fuse, generate) with request/session correlation IDs.
- **Metrics:** Prometheus histograms/counters for latency, errors, decisions, circuit-breaker state, model load, embedding latency, fusion output.
- **Logging:** structlog structured JSON with event names and session IDs.
- **Resilience:** per-service circuit breakers (Qdrant/Redis/Mongo) with half-open probing; tenacity retries with exponential backoff; graceful degradation fallbacks at every tool boundary.
- **Health:** infra_registry.health_check + model_loader.health_check expose readiness; main.py runs staggered warmup on startup.

Gap (Phase 31 — Monitoring): wire Prometheus to Grafana dashboards and an OTel collector; add RAG-quality SLOs (recall@k, faithfulness) as first-class monitored signals, not just system metrics.

9. Recommended Target Folder Structure

The current layout works but mixes concerns. The structure below (applied in Phase 24) clarifies boundaries and is the one the dev-team skills will enforce. Names preserved where possible to minimize churn.

```
multimodal-rag-assistant/
|-- app/
| |-- api/ routes, schemas, deps, middleware (auth P27)
| |-- agents/ controller, router, planner, executor, registry
| |-- ingestion/ per-modality loaders + router + chunker
| |-- embeddings/ text, clip, image, multimodal
| |-- retrieval/ bm25, dense, hybrid, reranker
| |-- reasoning/ decomposer, fusion, reasoning_engine
| |-- memory/ redis, mongo, manager, filter, fusion, summarizer
| |-- llm/ gguf_model, prompt_builder
| |-- tools/ web_search, (future tools)
| |-- pipeline/ ingestion_pipeline, query_pipeline, rag_pipeline
| |-- core/ config, infra_registry, model_loader, metrics
| |-- eval/ (P25) ragas harness, datasets, gates
| |-- guardrails/ (P26) input/output validators, jailbreak clf
| |-- auth/ (P27) users, JWT/OAuth, tenant isolation
| '-- utils/ logger, helpers
-- ui/ (P28) gradio_app
-- deploy/ (P30) Dockerfile, compose, terraform, k8s
-- .github/workflows/ (P29) ci.yml, eval-gate.yml, cd.yml
-- monitoring/ (P31) prometheus, grafana dashboards, otel
-- scripts/ init_qdrant, download_model, seed
-- tests/ unit, integration, eval
-- docs/ this design + ADRs + runbook
'-- pyproject.toml .env.example README CHANGELOG VERSION
```

10. Roadmap Snapshot (Phases 24 -> 31)

Phase	Name	Outcome	Status
24	Production-Grade Upgrade	Hardening, structure, resilience	IN PROGRESS
25	Evaluation	Ragas harness, recall@k/faithfulness, CI gate	NEXT
26	Guardrails	Input/output validation, jailbreak & PII egress	PLANNED
27	Authentication	Multi-user, JWT/OAuth, per-tenant isolation	PLANNED
28	Gradio UI	Multimodal chat UI with streaming + sources	PLANNED
29	MLOps / CI-CD	Pipelines, model registry, automated eval gate	PLANNED
30	AWS Deployment	Containerized cloud deploy + IaC	PLANNED
31	Monitoring	Grafana SLOs, alerting, RAG-quality dashboards	PLANNED

Detailed Phase 25–31 execution plan is delivered in the companion document (File 2). The dev-team automation to execute it is delivered as .claude skills (File 3).

Document status: System Design v2.0 — reconciled against 49 analyzed source files. This supersedes the original plan PDF where the implementation diverged. Authoritative for Phases 24–31.